

AIM - To implement Ridge Regression and Lasso Regression using Python and compare their performance with Linear Regression on a regression dataset.

Tools Required

- Python
- Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

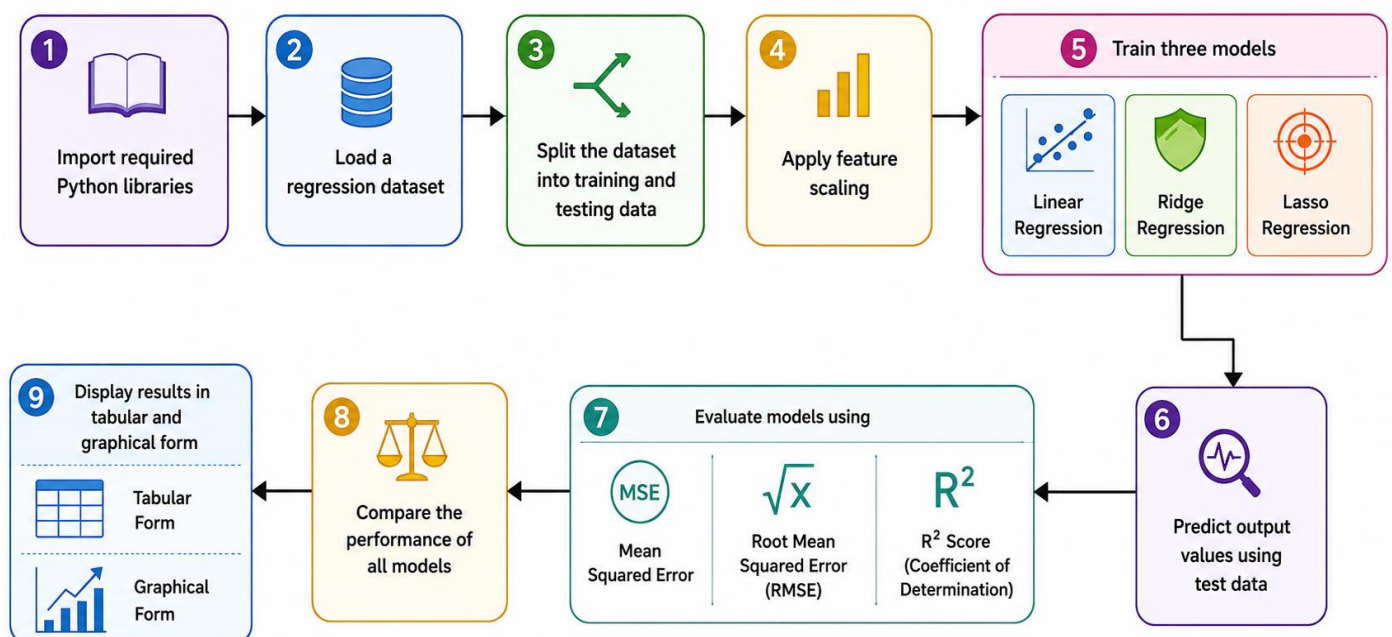
Theory

Linear Regression is a supervised learning algorithm used to predict continuous values. It tries to find the best-fit line by minimizing the difference between actual and predicted values..

Ridge Regression adds an L2 penalty to the loss function. It reduces the magnitude of coefficients but usually does not make them exactly zero. Scikit-learn describes Ridge as linear least squares with L2 regularization.

Lasso Regression adds an L1 penalty to the loss function. It can shrink some coefficients exactly to zero, so it also helps in feature selection. Scikit-learn notes that Lasso estimates sparse coefficients using L1 regularization.

Methodology



Code :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
print("Dataset shape:", X.shape)
print("Features:", diabetes.feature_names)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

linear = LinearRegression()
ridge = Ridge(alpha=1.5)
lasso = Lasso(alpha=1.5)

linear.fit(X_train, y_train)
ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)

linear_pred = linear.predict(X_test)
ridge_pred = ridge.predict(X_test)
lasso_pred = lasso.predict(X_test)
```

```

models = {
    "Linear Regression": linear_pred,
    "Ridge Regression": ridge_pred,
    "Lasso Regression": lasso_pred
}

results = []

for name, predictions in models.items():
    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)
    results.append([name, mse, rmse, r2])

results_df = pd.DataFrame(results, columns=["Model", "MSE", "RMSE", "R2 Score"])
results_df

models = results_df["Model"].tolist()
mse_values = results_df["MSE"].tolist()
plt.figure(figsize=(6,4))
plt.bar(models, mse_values)
plt.xlabel("Models")
plt.ylabel("MSE")
plt.title("Error Comparison")
plt.show()

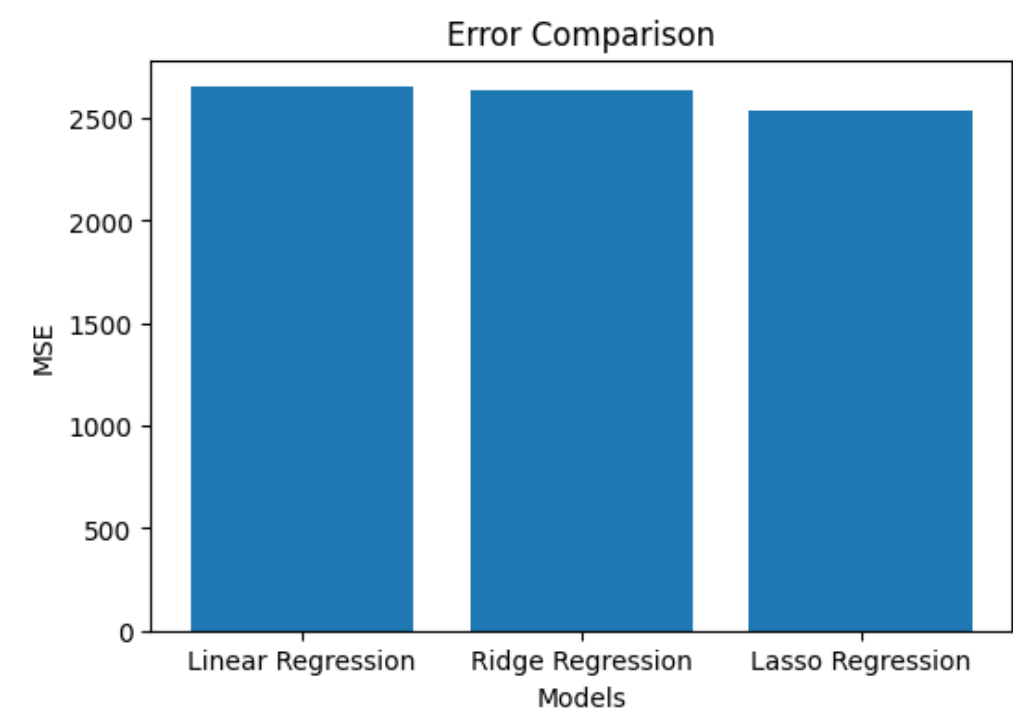
best_rmse = results_df.loc[results_df["RMSE"].idxmin(), "Model"]
best_r2 = results_df.loc[results_df["R2 Score"].idxmax(), "Model"]

print("Best model based on RMSE:", best_rmse)
print("Best model based on R2 Score:", best_r2)

```

Outputs:

	Model	MSE	RMSE	R2 Score
0	Linear Regression	2903.010967	53.879597	0.551818
1	Ridge Regression	2917.450541	54.013429	0.549589
2	Lasso Regression	2957.243154	54.380540	0.543446



Best model based on RMSE: Linear Regression
Best model based on R2 Score: Linear Regression

Result

- Linear Regression achieved the lowest MSE (2903.01) and lowest RMSE (53.88), indicating the smallest prediction error.
- It also obtained the highest R² Score (0.5518), meaning it explained approximately 55.18% of the variance in the target variable.
- Ridge and Lasso Regression performed similarly but showed slightly higher errors due to the effect of regularization.

Learning Outcome

- Understood the implementation of Linear Regression, Ridge Regression, and Lasso Regression models. Learned how to preprocess data by splitting it into training and testing sets and applying feature scaling.
- Gained experience in evaluating regression models using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R² Score.

